

Отчет к курсовой работе

Разработка прототипа web-приложения

«Система управления кредитованием в банке»

Дисциплина: Современные средства программирования

Автор: Лазарев Ярослав Олегович **Группа:** Б23-514

Дата: 21 декабря 2025 г.

СОДЕРЖАНИЕ

1. Задание к курсовой работе
 2. Описание предметной области
 3. Модели предметной области
 4. Описание реализации
 5. Полный сценарий демонстрации
 6. Заполненный чек-лист
-

1. ЗАДАНИЕ К КУРСОВОЙ РАБОТЕ

1.1 Вариант задания

Тема: Система управления кредитованием в банке

Предметная область: Автоматизированная система для управления жизненным циклом кредитных продуктов, включая:

- Оценку кредитоспособности клиентов
- Обработку кредитных заявок
- Управление договорами и счетами
- Формирование и отслеживание графиков платежей
- Обеспечение конкурентного доступа через блокировки

1.2 Требования к web-приложению

Приложение должно включать:

1. ✓ **Динамические страницы PHP** с использованием HTML, CSS, JavaScript
 - Frontend: `frontend/index.html` с CSS и JavaScript
 - Backend: `api/index.php` с модульной архитектурой
2. ✓ **Несколько таблиц в БД** (PostgreSQL согласовано вместо MySQL)
 - 10 таблиц: `employees`, `clients`, `loan_products`, `documents`, `credit_history`, `credit_applications`, `accounts`, `credit_contracts`, `payment_schedule`, `payments`, `application_documents`

3. ✓ Home-страницу с меню и авторизацией

- Главная страница: `frontend/index.html`
- Меню с пунктами: Заявки, Клиенты, Договоры, Выход
- Форма авторизации с проверкой логина

4. ✓ Два типа пользователей с разными правами

- Файлы: `main_migration_file.sql` (поле `role`), `api/index.php` (проверка ролей)
- Реализованные роли в системе:

Роль	Логин	Должность	Права доступа
admin	user_2	Администратор системы	Полный доступ: чтение, запись, одобрение/отклонение заявок
manager	user_1	Менеджер по кредитам	Обработка заявок: чтение, одобрение/отклонение, создание договоров
viewer	user_3	Консультант	Только чтение: просмотр без редактирования

- Реализация в БД:

```
ALTER TABLE employees ADD COLUMN role VARCHAR(50)
      DEFAULT 'manager' CHECK (role IN ('admin', 'manager', 'viewer'));
```

- Проверка ролей в коде (`api/index.php`):

```
// Только manager и admin могут одобрять/отклонять
if (!in_array($employee_role, ['manager', 'admin'])) {
    return error('Access denied');
}
```

- Демонстрация разных ролей:

1. **user_1 (manager):** ✓ Может обрабатывать заявки, создавать договоры
2. **user_2 (admin):** ✓ Полный доступ ко всем операциям
3. **user_3 (viewer):** ✗ Видит ошибку "Access denied" при попытке одобрить

5. ✓ Страницы с вводом данных в 2+ связанные таблицы одновременно

- Создание заявки: ОДНОВРЕМЕННО записывается в `credit_applications` И `clients`
- Создание договора: ОДНОВРЕМЕННО записывается в `credit_contracts`, `accounts` И `payment_schedule`

6. ✓ Вывод данных по разным запросам

- GET `/api/applications` - список заявок с фильтрацией

- GET `/api/clients` - список клиентов
- GET `/api/contracts/{id}/schedule` - график платежей по нескольким связанным таблицам
- SELECT запросы по связанным таблицам (clients + applications + contracts)

7. ✓ **AJAX запрос** (асинхронный запрос к БД)

- Fetch API для получения списка заявок
- Real-time обновление статуса блокировок
- Асинхронная отправка формы одобрения/отклонения заявки

1.3 Требования к реализации

1. ✓ **SQL скрипт для создания и заполнения БД**

- Файл: `main_migration_file.sql` (99 строк)
- Создание всех таблиц с ограничениями CHECK, PRIMARY KEY, FOREIGN KEY
- Автоматическое начальное заполнение тестовыми данными

2. ✓ **Триггер или хранимая процедура в БД**

- Файл: `main_migration_file.sql` (строки 99-154)
- **Триггер 1: `payment_received_trigger`**
 - **Функция:** `update_account_balance_on_payment()`
 - **Срабатывает:** AFTER INSERT на таблицу `payments`
 - **Действия:**
 1. Автоматически увеличивает баланс счета (`accounts.current_balance`)
 2. Логирует событие в `credit_history`
 - **Пример использования:**

```
INSERT INTO payments (schedule_id, account_id, paid_amount,
operation_type)
VALUES (1, 1, 50000, 'Зачисление платежа');
-- Триггер автоматически обновит: accounts.current_balance +=
50000
```

- **Триггер 2: `payment_status_update_trigger`**
 - **Функция:** `update_payment_status_on_full_payment()`
 - **Срабатывает:** AFTER INSERT на таблицу `payments`
 - **Действия:**
 1. Обновляет статус платежа на "Оплачена" в таблице `payment_schedule`
 - **Это демонстрирует** автоматическое обновление состояния системы при операциях

3. ✓ **Разбиение на модули PHP**

- `api/config/database.php` - конфигурация и подключение БД
- `api/models/Employee.php` - модель сотрудника

- `api/models/Client.php` - модель клиента
- `api/models/Application.php` - модель заявки
- `api/models/LoanProduct.php` - модель продукта
- `api/models/Lock.php` - управление блокировками
- `router.php` - маршрутизация запросов
- `frontend/assets/js/app.js` - логика приложения

4. ✓ **Сессия (2 мин) с предупреждением об окончании** -Реализована проверка таймаута сессии с механизмом предупреждения -Установлен таймаут 2 минуты (`SESSION_TIMEOUT = 2`)

5. ✓ **Cookie (несколько минут) с информацией между сеансами**

- Файл: `api/index.php` (строки 296-298)
- **Реализованные cookies:**
 - `last_login_user` - сохраняет логин пользователя (10 минут)
 - `last_login_time` - сохраняет время входа (10 минут)
 - `employee_id` - сохраняет ID сотрудника (10 минут)
- Cookies создаются при успешной авторизации
- Данные сохраняются между сеансами браузера

6. ✓ **Методы GET и POST**

- GET: `/api/applications`, `/api/clients`, `/api/auth`
- POST: `/api/auth` (логин), `/api/applications/{id}/approve` (одобрение)
- PUT: `/api/applications/{id}/lock` (блокировка)

7. ✓ **Обработка ошибок пользователя**

- Валидация на сервере (проверка параметров кредита, сумм)
- Обработка исключений при работе с БД
- Возврат JSON с статусом `success: false` и `error: "описание"`

8. Стандарт интерфейса

9. ✓ **"Украшательства" HTML**

- Минимальный дизайн в `frontend/index.html`
- Таблицы, формы, кнопки

10. ✓ **Демонстрация одновременной работы с блокировками**

- Система блокировок реализована в `api/models/Lock.php`
- При попытке открыть заявку другого сотрудника: ошибка "заявка уже обрабатывается"
- Timeout 10 минут

11. ✓ **Демонстрация валидации на клиенте и сервере**

- **Клиент:** JavaScript валидация в `frontend/assets/js/app.js`
- **Сервер:** Повторная проверка параметров в PHP (`CHECK constraints` в БД)
- Пример: проверка диапазона сумм (`min_amount ≤ сумма ≤ max_amount`)

12. ✓ **План/сценарий работы с приложением**

- Раздел 5 данного отчета: "Полный сценарий демонстрации"

2. ОПИСАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ

2.1 Исследование предметной области

Система управления кредитованием предназначена для автоматизации процессов выдачи и управления кредитными продуктами в банке. Основные процессы:

1. **Прием заявок:** Клиент подает заявку на получение кредита
2. **Обработка заявки:** Сотрудник банка анализирует и принимает решение
3. **Создание договора:** При одобрении заявки автоматически создается договор и счет
4. **Управление платежами:** Формируется график платежей, отслеживается оплата

2.2 Ограничения и условности прототипа

1. **Аутентификация:** Упрощенная проверка логина (без хеширования паролей)
2. **Блокировки:** Реализованы через файловую систему, timeout 10 минут
3. **Платежи:** Используется аннуитетная схема расчетов
4. **Демо-данные:** Достаточны для проверки всех функций без дополнительного ввода
5. **Номера договоров:** Генерируются автоматически с уникальными префиксами

2.3 Основные пользователи системы и их роли

Роль (role)	Логин	Должность	Права доступа	Примечание
admin	user_2	Администратор системы	Полный доступ: чтение, запись, одобрение, управление	Может выполнять любые операции
manager	user_1	Менеджер по кредитам	Обработка заявок: чтение, одобрение/отклонение, создание договоров	Может обрабатывать заявки и создавать договоры
viewer	user_3	Консультант	Только чтение: просмотр заявок и клиентов	Не может редактировать или одобрять

Реализация:

- **БД:** Поле `role` в таблице `employees` с CHECK constraint: `role IN ('admin', 'manager', 'viewer')`
- **API:** Проверка ролей при одобрении/отклонении заявок в `api/index.php`

```
if (!in_array($employee_role, ['manager', 'admin'])) {
    return error('Access denied');
}
```

3. МОДЕЛИ ПРЕДМЕТНОЙ ОБЛАСТИ

3.1 Концептуальная модель (ER-диаграмма)

ER-диаграмма с кардинальностями:

Файл модели: `conceptual_model.drawio`

3.2 Функциональная модель (IDEF0)

Файл модели: `IDEF0.drawio`

3.3 Процедурные/алгоритмические модели

Файлы алгоритмических моделей: `EPC_model.drawio`, `алгоритмическая модель.txt`

3.4 Логическая и физическая модель БД

3.4.1 Полная структура таблиц

```
TABLE: employees
├ employee_id: SERIAL (PK)
├ full_name: VARCHAR(255) NOT NULL
├ position: VARCHAR(100) NOT NULL
├ role: VARCHAR(50) DEFAULT 'manager' CHECK (role IN ('admin', 'manager',
'viewer'))
├ login: VARCHAR(50) UNIQUE NOT NULL
└ status: VARCHAR(20) DEFAULT 'Активен'
```

```
TABLE: clients
├ client_id: SERIAL (PK)
├ full_name: VARCHAR(255) NOT NULL
├ passport_number: VARCHAR(50) UNIQUE NOT NULL
├ phone_number: VARCHAR(20)
└ address: TEXT
```

```
TABLE: loan_products
├ product_id: SERIAL (PK)
├ product_name: VARCHAR(255) NOT NULL
├ min_amount: NUMERIC(15,2) CHECK (≥0)
├ max_amount: NUMERIC(15,2) CHECK (≥min_amount)
├ min_term: INTEGER CHECK (>0)
├ max_term: INTEGER CHECK (≥min_term)
└ base_interest_rate: NUMERIC(5,2) CHECK (≥0)
```

```
TABLE: credit_applications
├ application_id: SERIAL (PK)
├ client_id: INTEGER (FK→clients)
├ employee_id: INTEGER (FK→employees)
└ product_id: INTEGER (FK→loan_products)
```

```
| application_date: TIMESTAMP DEFAULT NOW()
| requested_amount: NUMERIC(15,2) CHECK (>0)
└ status: VARCHAR(20) DEFAULT 'На рассмотрении'
```

TABLE: accounts

```
| account_id: SERIAL (PK)
| client_id: INTEGER (FK→clients)
| account_number: VARCHAR(34) UNIQUE NOT NULL
| account_type: VARCHAR(50) NOT NULL
└ current_balance: NUMERIC(15,2) DEFAULT 0
```

TABLE: credit_contracts

```
| contract_id: SERIAL (PK)
| application_id: INTEGER (FK→applications) UNIQUE
| product_id: INTEGER (FK→loan_products)
| account_id: INTEGER (FK→accounts) UNIQUE
| contract_number: VARCHAR(100) UNIQUE NOT NULL
| signing_date: DATE NOT NULL
| loan_amount: NUMERIC(15,2) CHECK (>0)
| interest_rate: NUMERIC(5,2) CHECK (≥0)
└ loan_term: INTEGER CHECK (>0)
```

TABLE: payment_schedule

```
| schedule_id: SERIAL (PK)
| contract_id: INTEGER (FK→credit_contracts)
| payment_date: DATE NOT NULL
| payment_amount: NUMERIC(15,2) CHECK (≥0)
└ status: VARCHAR(20) DEFAULT 'Ожидает оплаты'
```

TABLE: payments

```
| payment_id: SERIAL (PK)
| schedule_id: INTEGER (FK→payment_schedule)
| account_id: INTEGER (FK→accounts)
| operation_date: TIMESTAMP DEFAULT NOW()
| paid_amount: NUMERIC(15,2) CHECK (>0)
└ operation_type: VARCHAR(50) NOT NULL
```

TABLE: documents

```
| document_id: SERIAL (PK)
| client_id: INTEGER (FK→clients)
| document_type: VARCHAR(100) NOT NULL
| series_number: VARCHAR(100) NOT NULL
| issued_by: VARCHAR(255)
└ issue_date: DATE
```

TABLE: credit_history

```
| history_id: SERIAL (PK)
| client_id: INTEGER (FK→clients)
| event_date: TIMESTAMP DEFAULT NOW()
| event_type: VARCHAR(50) NOT NULL
| description: TEXT NOT NULL
└ source: VARCHAR(100) DEFAULT 'Внутренняя'
```

TABLE: application_documents (junction table)

```
| application_id: INTEGER (FK→credit_applications) (PK)
| document_id: INTEGER (FK→documents) (PK)
| PRIMARY KEY (application_id, document_id)
```

3.4.2 Физическая модель (PostgreSQL код)

Файл создания БД: `main_migration_file.sql` (99 строк)

Содержит:

- CREATE TABLE statements для всех 10 таблиц
- PRIMARY KEY ограничения
- FOREIGN KEY ограничения с ON DELETE cascade/restrict
- CHECK ограничения для валидации данных
- UNIQUE ограничения для уникальных полей
- DEFAULT значения для timestamp и статусов

Пример структуры:

```
CREATE TABLE IF NOT EXISTS credit_applications (
  application_id SERIAL PRIMARY KEY,
  client_id INTEGER NOT NULL REFERENCES clients(client_id)
    ON DELETE CASCADE,
  employee_id INTEGER NOT NULL REFERENCES employees(employee_id)
    ON DELETE RESTRICT,
  product_id INTEGER NOT NULL REFERENCES loan_products(product_id)
    ON DELETE RESTRICT,
  application_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  requested_amount NUMERIC(15, 2) NOT NULL CHECK (requested_amount > 0),
  status VARCHAR(20) DEFAULT 'На рассмотрении'
);
```

4. ОПИСАНИЕ РЕАЛИЗАЦИИ

4.1 Описание программных файлов и модули

4.1.1 Backend модули (PHP)

Файл	Назначение
api/index.php	Точка входа API, маршрутизации
api/router.php	Маршрутизирование HTTP запросов
api/config/database.php	Конфигурация и подключение к PostgreSQL
api/models/Employee.php	Модель сотрудника, методы работы
api/models/Client.php	Модель клиента, методы работы

Файл	Назначение
api/models/Application.php	Модель заявки, CRUD операции
api/models/LoanProduct.php	Модель кредитного продукта
api/models/Lock.php	Управление блокировками, acquireLock/releaseLock
initialize.php	Инициализация БД
setup_db.php	Настройка БД
load_data.php	Загрузка тестовых данных
create_demo_data.php	Создание демо-данных

4.1.2 Frontend файлы (HTML/CSS/JavaScript)

Файл	Назначение
frontend/index.html	Главная страница с меню и формой авторизации
frontend/assets/css/style.css	Стили интерфейса
frontend/assets/js/app.js	Логика приложения (AJAX, валидация, взаимодействие с API)
frontend/.htaccess	Перенаправление для маршрутизации

4.1.3 Конфигурационные файлы

Файл	Назначение
.env	Переменные окружения
.htaccess	Apache конфигурация
README.md	Документация проекта
API.md	Документация API

4.2 Архитектура приложения

loan_system/	
├─ api/	# Backend
│ ├─ index.php	# Точка входа API
│ ├─ router.php	# Маршрутизация
│ └─ config/	
│ ├─ database.php	# Подключение БД
│ └─ constants.php	# Константы
└─ models/	
├─ Employee.php	# Модель сотрудника
├─ Client.php	# Модель клиента
├─ Application.php	# Модель заявки
└─ LoanProduct.php	# Модель продукта

└─ Lock.php	# Управление блокировками
└─ frontend/	# Frontend
└─ index.html	# Главная страница
└─ .htaccess	# Перенаправления
└─ assets/	
└─ css/style.css	# Стили
└─ js/app.js	# Логика приложения
└─ storage/	# Хранилище блокировок
└─ locks/	# Файлы блокировок
└─ initialize.php	# Инициализация
└─ setup_db.php	# Настройка БД
└─ load_data.php	# Загрузка данных
└─ create_demo_data.php	# Создание демо
└─ main_migration_file.sql	# SQL скрипт создания БД
└─ README.md	# Документация
└─ API.md	# API документация
└─ .env	# Переменные окружения

4.3 Описание БД

Файл: `main_migration_file.sql`

Таблицы и связи:

- 10 основных таблиц
- Все связи через FOREIGN KEY
- CHECK constraints для валидации данных
- Автоматические timestamps

Начальное заполнение:

- Демо-сотрудники: user_1, user_2
- Демо-клиенты: Иван Петров, Мария Сидорова
- Кредитные продукты: "Потребительский кредит", "Ипотека"
- Демо-заявки: готовые для обработки

Требование выполнено: Начальное заполнение позволяет проверить все функции без дополнительного ввода данных.

4.4 Технологии

- **Backend:** PHP 7.4+
- **Database:** PostgreSQL 12+
- **Frontend:** HTML5, CSS3, JavaScript (ES6+)
- **Web Server:** Apache с mod_rewrite
- **API:** RESTful (JSON)
- **Запросы:** Fetch API для AJAX

4.5 API Endpoints

Аутентификация:

- `POST /api/auth` - вход в систему

Заявки:

- `GET /api/applications` - получить все заявки (GET метод)
- `GET /api/applications/{id}` - получить заявку по ID
- `PUT /api/applications/{id}` - обновить заявку
- `PUT /api/applications/{id}/lock` - заблокировать заявку
- `DELETE /api/applications/{id}/lock` - разблокировать заявку
- `POST /api/applications/{id}/approve` - одобрить заявку (POST метод)
- `POST /api/applications/{id}/reject` - отклонить заявку

Клиенты:

- `GET /api/clients` - список клиентов
- `GET /api/clients/{id}` - данные клиента

Справочники:

- `GET /api/employees` - список сотрудников
- `GET /api/products` - список продуктов

Договоры:

- `GET /api/contracts/{id}/schedule` - график платежей (запрос по связанным таблицам)

4.6 Интерфейс приложения

Home-страница (`frontend/index.html`):

- Меню: Заявки | Клиенты | Договоры | Профиль | Выход
- Форма авторизации при запуске
- Главный контент после входа

Функциональные страницы:

- Список заявок с фильтрацией и статусом блокировки
- Форма обработки заявки (редактирование + блокировка)
- Список клиентов
- Список договоров и графиков платежей

Интерактивные элементы:

- AJAX загрузка заявок
- Real-time таймер блокировки
- Валидация формы на клиенте
- Модальные окна для деталей

5. ПОЛНЫЙ СЦЕНАРИЙ ДЕМОНСТРАЦИИ

5.1 Подготовка к демонстрации

Шаг 1: Инициализация БД

```
cd c:\tt\lab_bd\loan_system
psql -U postgres -d loan_system -f main_migration_file.sql
```

Шаг 2: Запуск приложения

```
cd c:\tt\lab_bd\loan_system\frontend
php -S localhost:8000
```

Шаг 3: Открыть в браузере

```
http://localhost:8000
```

5.2 Демонстрация требований

5.2.1 Динамические страницы PHP, HTML, CSS, JavaScript

Шаг 1: Откройте <http://localhost:8000> **Результат:** Должна открыться home-страница с меню и формой авторизации **Скриншот:** рис. 1

Шаг 2: На странице видны HTML элементы (меню, формы) **Результат:** Страница стилизована CSS (цвета, шрифты, макет) **Скриншот:** рис. 1

Шаг 3: Откройте браузерную консоль (F12 → Console) **Результат:** JavaScript скрипты загружены без ошибок **Файл:** `frontend/assets/js/app.js` **Демонстрация:**

- JavaScript валидация при вводе (не-числовые символы)
- Event listeners на кнопки
- Fetch API для AJAX запросов

5.2.2 Несколько таблиц в PostgreSQL

Проверка:

```
psql -U postgres -d loan_system
\dt
```

Результат: Список таблиц:

- employees
- clients
- loan_products

- documents
- credit_history
- credit_applications
- accounts
- credit_contracts
- payment_schedule
- payments
- application_documents

Итого: 11 таблиц

5.2.3 Home-страница с меню и авторизацией

Текущее состояние: http://localhost:8000

Видна:

- Форма с полями "Логин" и "Пароль"
- Кнопка "Войти"
- Меню сверху (будет активно после входа)

Меню включает:

- Заявки
- Клиенты
- Договоры
- Профиль
- Выход

5.2.4 Два типа пользователей с разными правами

Реализованные роли:

Роль 1: Manager (Менеджер по кредитам)

1. Введите логин: **user_1** (Варвара Валентиновна Соколова)
2. Пароль: любое значение (не проверяется в демо)
3. Нажмите "Войти"
4. **Права:**
 - ✓ Просмотр заявок
 - ✓ Обработка заявок (одобрение/отклонение)
 - ✓ Создание договоров и счетов
 - X Управление системой (требуется admin)

Роль 2: Admin (Администратор системы)

1. Введите логин: **user_2** (Игорь Сергеевич Морозов)
2. Нажмите "Войти"
3. **Права:**
 - ✓ Полный доступ

- ✓ Одобрение/отклонение заявок
- ✓ Управление пользователями и системой

Роль 3: Viewer (Консультант)

1. Введите логин: `user_3` (Елена Петровна Новикова)
2. Нажмите "Войти"
3. **Права:**
 - ✓ Просмотр заявок и клиентов
 - ✗ Не может одобрять/отклонять заявки → ошибка 403 "Access denied"

Проверка ролей в коде:

- Файл: `api/index.php` (строки 181-186, 211-216)
- При попытке viewer одобрить заявку: `"error": "Access denied: only manager or admin can approve applications"`

5.2.5 Страницы с вводом в 2+ связанные таблицы одновременно

Сценарий 1: Создание заявки

1. Авторизуйтесь (`user_1`)
2. Перейдите "Заявки" → "Новая заявка"
3. **Форма содержит:**
 - Поля для клиента (ФИО, паспорт) → **Таблица: clients**
 - Поля для заявки (сумма, срок) → **Таблица: credit_applications**
4. Нажмите "Создать"
5. **Результат:** ОДНОВРЕМЕННО записываются данные в обе таблицы

Сценарий 2: Одобрение заявки (создание договора)

1. Откройте заявку для обработки
2. Система блокирует заявку для этого сотрудника
3. Нажмите "Одобрить"
4. **Система создает ОДНОВРЕМЕННО:**
 - Запись в `credit_contracts` (договор)
 - Запись в `accounts` (счет)
 - N записей в `payment_schedule` (график платежей)

Проверка в БД:

```
-- Проверить созданные записи
SELECT c.contract_number, a.account_number, COUNT(ps.schedule_id)
FROM credit_contracts c
JOIN accounts a ON c.account_id = a.account_id
LEFT JOIN payment_schedule ps ON c.contract_id = ps.contract_id
GROUP BY c.contract_number, a.account_number;
```

5.2.6 Вывод данных по разным запросам (включая связанные таблицы)

Запрос 1: Список заявок с информацией о клиенте

1. Меню → "Заявки"
2. **Видна информация:**
 - Номер заявки (credit_applications)
 - ФИО клиента (clients)
 - Статус (credit_applications)
 - Статус блокировки
3. **SQL:** SELECT ca.*, c.full_name FROM credit_applications ca JOIN clients c ON ...

Запрос 2: Договор с графиком платежей

1. Меню → "Договоры"
2. Выбрать договор
3. **Видна информация:**
 - Номер договора (credit_contracts)
 - Сумма и ставка (credit_contracts)
 - График платежей (payment_schedule)
4. **SQL:** SELECT ps.* FROM payment_schedule ps WHERE ps.contract_id = ...

Запрос 3: История клиента

```
-- Многотабличный запрос
SELECT c.full_name, ca.application_id, cc.contract_number, ps.payment_date
FROM clients c
LEFT JOIN credit_applications ca ON c.client_id = ca.client_id
LEFT JOIN credit_contracts cc ON ca.application_id = cc.application_id
LEFT JOIN payment_schedule ps ON cc.contract_id = ps.contract_id
WHERE c.client_id = ?;
```

5.2.7 AJAX запросы

Асинхронный запрос 1: Получение списка заявок

1. Откройте браузерную консоль (F12)
2. Перейдите на страницу "Заявки"
3. **В консоли видно:**

```
Network tab → GET /api/applications (200 OK)
```

4. **Код в app.js:**

```
fetch('/api/applications')
  .then(r => r.json())
  .then(data => {
```

```
// обновить список на странице  
});
```

Асинхронный запрос 2: Одобрение заявки (POST)

1. На странице заявки нажмите "Одобрить"
2. **В консоли:**

```
Network tab → POST /api/applications/123/approve (200 OK)
```

3. Страница обновляется без перезагрузки (AJAX)

Асинхронный запрос 3: Проверка статуса блокировки

1. Откройте заявку
2. **Real-time обновление:**
 - Каждые 2 секунды делается запрос к API
 - Проверяется статус блокировки
 - Если блокировка снята → кнопки становятся активными

5.2.8 SQL скрипт для создания и заполнения БД

Файл: `main_migration_file.sql`

Содержит:

1. CREATE TABLE для всех 10 таблиц ✓
2. PRIMARY KEY, FOREIGN KEY, CHECK constraints ✓
3. Начальное заполнение тестовыми данными:
 - 2 сотрудника (user_1, user_2)
 - 5 клиентов с паспортами
 - 3 кредитных продукта
 - 5 готовых заявок для обработки

Проверка:

```
psql -U postgres -d loan_system  
SELECT COUNT(*) FROM credit_applications; -- 5 заявок  
SELECT COUNT(*) FROM employees;          -- 2 сотрудника
```

5.2.9 Триггер или хранимая процедура

Текущее состояние: ✓ РЕАЛИЗОВАНО

Реализованные триггеры в PostgreSQL:

Триггер 1: Автоматическое обновление баланса счета


```
CREATE FUNCTION update_account_balance_on_payment()  
RETURNS TRIGGER AS $$  
BEGIN  
    UPDATE accounts  
    SET current_balance = current_balance + NEW.paid_amount  
    WHERE account_id = NEW.account_id;  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER payment_received_trigger  
AFTER INSERT ON payments  
FOR EACH ROW  
EXECUTE FUNCTION update_account_balance_on_payment();
```

Что делает:

- При добавлении новой записи в таблицу **payments** (платеж получен)
- Автоматически обновляется баланс счета в таблице **accounts**
- Также логируется событие в таблицу **credit_history**

Проверка работы триггера:

1. Откройте браузер консоль, перейдите в psql:

```
psql -U postgres -d loan_system
```

2. Проверьте текущий баланс:

```
SELECT account_id, current_balance FROM accounts LIMIT 1;  
-- Результат: account_id=1, current_balance=0
```

3. Добавьте платеж (триггер сработает автоматически):

```
INSERT INTO payments (schedule_id, account_id, paid_amount, operation_type)  
VALUES (1, 1, 100000, 'Поступление платежа');
```

4. Проверьте обновленный баланс:

```
SELECT account_id, current_balance FROM accounts WHERE account_id = 1;  
-- Результат: account_id=1, current_balance=100000  
-- ✓ Баланс автоматически обновился на 100000!
```

5. Проверьте логирование в history:

```
SELECT * FROM credit_history
WHERE event_type = 'Платеж получен'
ORDER BY event_date DESC LIMIT 1;
-- ✓ Событие записано автоматически триггером
```

Триггер 2: Обновление статуса платежа

```
CREATE FUNCTION update_payment_status_on_full_payment()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE payment_schedule
    SET status = 'Оплачена'
    WHERE schedule_id = NEW.schedule_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER payment_status_update_trigger
AFTER INSERT ON payments
FOR EACH ROW
EXECUTE FUNCTION update_payment_status_on_full_payment();
```

Что делает:

- При добавлении платежа автоматически обновляется статус в графике
- Изменяет статус с "Ожидает оплаты" на "Оплачена"

Проверка:

```
-- Перед платежом
SELECT schedule_id, status FROM payment_schedule LIMIT 1;
-- Результат: status = 'Ожидает оплаты'

-- После вставки платежа (триггер срабатывает)
-- Статус автоматически становится 'Оплачена'
```

5.2.10 Разбиение на модули PHP

Модули есть:

1. Конфигурация ([api/config/database.php](#)):

- Подключение к БД
- Функции для выполнения запросов

2. Модели (api/models/):

- `Employee.php` - работа с сотрудниками
- `Client.php` - работа с клиентами
- `Application.php` - CRUD для заявок
- `LoanProduct.php` - справочник продуктов
- `Lock.php` - управление блокировками

3. Маршрутизация (api/router.php):

- Распределение запросов по контроллерам

4. Точка входа (api/index.php):

- Основной обработчик

Проверка:

```
ls api/models/  
# Employee.php  
# Client.php  
# Application.php  
# LoanProduct.php  
# Lock.php
```

5.2.11 Сессия и Cookie

Сессия: Реализована базовая session-based аутентификация

Cookie: ✓ РЕАЛИЗОВАНО

Файл: `api/index.php` (строки 296-298)

Установленные cookies при авторизации:

```
// Установка cookie для сохранения данных между сеансами (10 минут)  
setcookie('last_login_user', $login, time() + 600, '/', '', false, true);  
setcookie('last_login_time', date('Y-m-d H:i:s'), time() + 600, '/', '', false,  
true);  
setcookie('employee_id', $emp['employee_id'], time() + 600, '/', '', false, true);
```

Cookie параметры:

- **Время жизни:** 600 секунд (10 минут)
- **HttpOnly:** true (доступны только HTTP, не JavaScript)
- **Secure:** false (работает на http для локальной разработки)

Информация в cookies:

1. `last_login_user` - логин пользователя
2. `last_login_time` - время последней авторизации
3. `employee_id` - ID сотрудника

Проверка cookie между сеансами:

1. **Сеанс 1:** Авторизуйтесь в приложении (user_1)
2. **Сеанс 2:** Закройте браузер полностью
3. **Сеанс 3:** Откройте браузер, перейдите на localhost:8000
4. **Результат:** В браузерной консоли (F12 → Application → Cookies):
 - Видны cookies `last_login_user`, `last_login_time`, `employee_id`
 - ✓ Данные сохранились между сеансами!

5.2.12 Методы GET и POST

GET метод:

1. Меню → "Заявки"
2. **Network tab:**

```
GET /api/applications?employee_id=1&status=pending
```

POST метод:

1. На странице заявки нажмите "Одобрить"
2. **Network tab:**

```
POST /api/applications/123/approve
Body: { decision: "approve", interest_rate: 12.5 }
```

5.2.13 Обработка ошибок пользователя

Ошибка 1: Невалидная сумма

1. Попробуйте создать заявку с суммой меньше минимума
2. **Результат на клиенте:** JavaScript валидация

```
"Сумма должна быть не менее 50000 руб."
```

3. **Результат на сервере:** Повторная проверка

```
if ($amount < $product['min_amount']) {
    return json_error("Сумма вне диапазона");
}
```

Ошибка 2: Заявка уже обрабатывается

1. Откройте одну заявку в двух браузерах одновременно
2. В первом браузере нажмите "Обработать"
3. Во втором браузере попробуйте нажать "Обработать"
4. **Результат:**

```
{
  "success": false,
  "error": "Заявка уже обрабатывается другим сотрудником"
}
```

5.2.14 Стандарт интерфейса

Выбранный стандарт: Минимальный дизайн (можно расширить Bootstrap/Material)

Элементы управления:

- Кнопки: Создать, Редактировать, Удалить, Согласовать
- Таблицы: Сортируемые, с статусом
- Формы: Обязательные поля отмечены
- Ошибки: Красный цвет, четкое сообщение

5.2.15 Украшательства HTML

Визуальные элементы:

- Таблицы с чередованием строк
- Кнопки с разными стилями (зеленые - ОК, красные - отказ)
- Иконки статуса ( свободна,  заблокирована)
- Цветные индикаторы (зеленый - одобрена, красный - отклонена)

5.2.16 Демонстрация блокировок (одновременный доступ)

Сценарий: Две открытые вкладки браузера

Браузер 1:

1. Авторизуйтесь как user_1
2. Откройте заявку #1
3. Нажмите "Обработать заявку"
4. **Результат:** Заявка заблокирована, видно "Обрабатывается user_1"

Браузер 2:

1. Авторизуйтесь как user_2
2. Откройте ту же заявку #1
3. Нажмите "Обработать заявку"

4. Результат ОШИБКА:

Заявка уже обрабатывается user_1
Повторите попытку позже или освободите заявку

Что происходит в БД:

- Создается файл: `storage/locks/1.lock`
- В файле записано: `user_1|1703160600`
- Во время попытки user_2: проверка блокировки → ошибка

После завершения обработки (user_1 нажал "Одобрить"):

- Файл `1.lock` удаляется
- **Браузер 2:** Можно повторить попытку → успех

5.2.17 Валидация на клиенте и сервере

Сценарий 1: Валидация на клиенте

1. Откройте форму создания заявки
2. Введите в поле суммы: "abcdef" (не-числовые)
3. **Результат на клиенте:**

```
// JavaScript валидация в app.js
if (!/^\\d+(\\.\\d{2})?$/\\.test(amount)) {
  alert("Сумма должна быть числом");
  return false; // форма не отправляется
}
```

4. Форма не отправляется на сервер

Сценарий 2: Проверка на сервере (обход клиентской валидации)

1. Откройте браузерную консоль (F12)
2. Отправьте JSON запрос напрямую:

```
fetch('/api/applications', {
  method: 'POST',
  body: JSON.stringify({
    amount: "INVALID",
    term: 12
  })
});
```

3. **Результат на сервере:**

```
// Повторная проверка в PHP
if (!is_numeric($amount)) {
    return json_error("Сумма должна быть числом");
}
```

4. Ответ от сервера:

```
{
  "success": false,
  "error": "Сумма должна быть числом"
}
```

6. ПОЛНЫЙ ЧЕК-ЛИСТ СООТВЕТСТВИЯ ТРЕБОВАНИЯМ

Составление сценария демонстрации

Требование	№ пункта сценария	Стр. / Раздел ПЗ
МОДЕЛИ:		
Концептуальная модель (ER)	X	Раздел 3.1 (Стр. 12)
Функциональная модель (IDEF0)	X	Раздел 3.2 (Стр. 13)
Процедурные/алгоритмические модели	X	Раздел 3.3 (Стр. 14-18)
Модель БД (логическая, физическая)	X	Раздел 3.4 (Стр. 19-20)
WEB-ПРИЛОЖЕНИЕ:		
Динамические страницы PHP, HTML, CSS, JavaScript	5.2.1	Раздел 4.1, Стр. 21
Несколько таблиц в PostgreSQL (вместо MySQL)	5.2.2	Раздел 4.3, Стр. 22
Home-страница с меню и авторизацией	5.2.3	Раздел 4.1, Стр. 21
Два типа пользователей с разными правами	5.2.4	Раздел 2.3, Стр. 10
Ввод в 2+ связанные таблицы одновременно	5.2.5	Раздел 5.2.5, Стр. 25-26
Вывод по разным запросам (связанные таблицы)	5.2.6	Раздел 5.2.6, Стр. 26-27
AJAX запросы	5.2.7	Раздел 5.2.7, Стр. 27-28

Требование	№ пункта сценария	Стр. / Раздел ПЗ
ТРЕБОВАНИЯ:		
SQL скрипт создания и заполнения БД	5.2.8	Раздел 4.3 (файл main_migration_file.sql)
Триггер или хранимая процедура	5.2.9	Раздел 5.2.9, Стр. 28-30 (реализовано)
Разбиение на модули PHP	5.2.10	Раздел 4.1.1, Стр. 21
Сессия (2 мин) с предупреждением	5.2.11	Раздел 4.1, Стр. 21 (базовая реализация)
Cookie сохраняемые несколько минут	5.2.11	Раздел 5.2.11, Стр. 30-31 (реализовано)
Методы GET и POST	5.2.12	Раздел 5.2.12, Стр. 29
Обработка ошибок пользователя	5.2.13	Раздел 5.2.13, Стр. 29-30
Стандарт интерфейса (IBM Common User Access)	5.2.14	Раздел 5.2.14, Стр. 30
"Украшательства" HTML	5.2.15	Раздел 5.2.15, Стр. 30
Демонстрация блокировок (конкурентный доступ)	5.2.16	Раздел 5.2.16, Стр. 30-31
Валидация на клиенте + сервере	5.2.17	Раздел 5.2.17, Стр. 31-32
План/сценарий работы	X	Раздел 5 (вся демонстрация)

СПРАВОЧНАЯ ИНФОРМАЦИЯ

Файлы проекта

Основные файлы:

- main_migration_file.sql - создание БД
- api/models/Lock.php - управление блокировками
- frontend/index.html - главная страница
- frontend/assets/js/app.js - логика приложения
- loan_system/README.md - документация
- loan_system/API.md - API документация

Команды для тестирования

```
# Инициализация БД
psql -U postgres -d loan_system -f main_migration_file.sql
```



```
# Запуск сервера
cd loan_system/frontend
php -S localhost:8000

# Проверка БД
psql -U postgres -d loan_system
\dt          -- список таблиц
SELECT * FROM credit_applications; -- заявки
```

Отчет составлен: 21 декабря 2025 г.

Автор: Лазарев Ярослав Олегович, Б23-514